

Объединение индексов (BitmapOr)

Когда возникает

Показать первые 20 самых старых «своих» или неназначенных заявок для обработки, причем свои в приоритете.

Как опознать

```
1  -> BitmapOr
2    -> Bitmap Index Scan
3    -> Bitmap Index Scan
```

Рекомендации

Использовать **UNION [ALL]** для объединения подзапросов по каждому из OR-блоков условий.

Пример:

```
1  CREATE TABLE tbl AS
2  SELECT
3    generate_series(1, 100000) pk  -- 100K "фактов"
4  , CASE
5    WHEN random() < 1::real/16 THEN NULL -- с вероятностью 1:16 запись
6    ELSE (random() * 100)::integer -- 100 разных внешних ключей
7    END fk_own;
8
9  CREATE INDEX ON tbl(fk_own, pk); -- индекс с "вроде как подходящей"
   сортировкой
10
11 SELECT
12   *
13 FROM
14   tbl
15 WHERE
16   fk_own = 1 OR -- свои
17   fk_own IS NULL -- ... или "ничьи"
18 ORDER BY
19   (fk_own = 1) DESC -- сначала "свои"
20 , pk
21 LIMIT 20;
```

#	node, ms	tree, ms	rows	ratio	node	sh.ht
	1.102	20			итоговые результаты	398
0	0.005	1.102	20	----	Limit	
1	0.162	1.097	20	49.9↑	-> Sort	
2	0.805	0.935	999	----	-> Bitmap Heap Scan on tbl	391
3	0.001	0.130		inf↑	-> BitmapOr	
4	0.124	0.124	999	2.0↓	-> Bitmap Index Scan on tbl_fk_own_pk_idx	5
5	0.005	0.005		inf↑	-> Bitmap Index Scan on tbl_fk_own_pk_idx	2

Исправляем:

```
1  (
2    SELECT
```

```

3      *
4  FROM
5      tbl
6  WHERE
7      fk_own = 1 -- сначала "свои" 20
8  ORDER BY
9      pk
10 LIMIT 20
11 )
12 UNION ALL
13 (
14     SELECT
15         *
16     FROM
17         tbl
18     WHERE
19         fk_own IS NULL -- потом "ничьи" 20
20     ORDER BY
21         pk
22     LIMIT 20
23 )
24 LIMIT 20; -- но всего - 20, больше и не надо

```

#	node	ms	tree	ms	rows	ratio	loops	node	sh.ht
		0.049		20				итоговые результаты	9
0	0.003	0.049	20	----				Limit	
1	0.003	0.046	20	2.01				-> Append	
2	0.002	0.043	20	----				-> Limit	
3	0.041	0.041	20	25.01				-> Index Only Scan using tbl_fk_own_pk_idx on tbl	9
4				inft	n/a			-> Limit	
5				inft	n/a			-> Sort	
6				inft	n/a			-> Bitmap Heap Scan on tbl tbl_1	
7				inft	n/a			-> Bitmap Index Scan on tbl_fk_own_pk_idx	

Мы воспользовались тем, что все 20 нужных записей были сразу получены уже в первом блоке, поэтому второй, с более «дорогим» Bitmap Heap Scan, даже не выполнялся — в итоге **в 22 раза быстрее, в 44 раза меньше чтений!**

Более детальный рассказ о данном способе оптимизации на конкретных примерах можно прочитать в статьях [PostgreSQL Antipatterns: вредные JOIN и OR](#) и [PostgreSQL Antipatterns: сказ об итеративной доработке поиска по названию, или «Оптимизация туда и обратно»](#).

Обобщенный вариант **упорядоченного отбора по нескольким ключам** (а не только по паре const/NULL) рассмотрен в статье [SQL HowTo: пишем while-цикл прямо в запросе, или «Элементарная трехходовка»](#).